VHD-FL: Verifiable Hierarchical Federated Learning with Distributed Key Generation for UAV Swarms

Zhongyang Shi, Hong Zhong, Qingyang Zhang, Ke Cheng, Fengqun Wang, and Jie Cui

***Abstract*—Federated learning (FL) over hierarchical unmanned aerial vehicle (UAV) swarms allows collaborative model training across worker UAVs, cluster-head UAVs, and a cloud server without relocating raw sensing data. Securing this architecture, however, imposes a set of coupled constraints that existing protocols do not jointly resolve: MCU-class worker UAVs cannot afford the number-theoretic transform underlying homomorphic encryption, no trusted authority is available to issue a group key, and the cluster-heads and cloud are mutually untrusted, may collude, and may tamper with encrypted inputs, fabricate the aggregation, or submit erroneous decryption shares. We present VHD-FL, a verifiable hierarchical FL framework that reconciles these constraints without introducing a new trusted entity. A cluster-head-layer distributed key generation produces the group key with zero participation by worker UAVs and zero key shares held by the cloud. A three-blind outsourced encryption protocol reuses a neighboring cluster-head as the third mask holder, yielding a 3-out-of-3 additive secret sharing that tolerates collusion among any two mask holders. Three-layer verifiability—an encryption zero-knowledge proof, a homomorphic hash over the aggregation, and a decryption proof—detects tampering, forged aggregation, and incorrect partial decryption against a malicious cloud. Experiments on MNIST, CIFAR-10, and FEMNIST show that VHD-FL matches plaintext FedAvg accuracy while reducing per-round worker cost to a single modular subtraction and one EdDSA signature, and that it tolerates up to $\tau - 1$ colluding cluster-heads and arbitrary worker dropouts.**

***Index Terms*—Federated learning, unmanned aerial vehicles, distributed key generation, homomorphic encryption, verifiability, collusion resistance.**

I. Introduction

UNMANNED aerial vehicle (UAV) swarms are increasingly deployed for disaster response, environmental monitoring, reconnaissance, and intelligent surveillance, in which a fleet of small, battery-powered aircraft collaborates to sense and act over a target area. Training a shared perception or decision model across the fleet is essential, yet the raw imagery, telemetry, and location traces are both volume-heavy and privacy-sensitive, which makes centralizing them impractical. Federated learning (FL) offers a natural fit: each vehicle trains locally on its own data and exchanges only model updates, so that raw samples never leave the device [1]. Because a flat, fully connected topology does not scale to large fleets, practical deployments organize the aircraft into a three-tier hierarchy in which worker UAVs are grouped into clusters governed by a more capable cluster-head UAV, and the cluster-heads in turn connect to a cloud server. Hierarchical aggregation over this topology reduces both the number of long-range links and the energy cost incurred at the weakest nodes [2], [3], and it has become the standard structure for UAV-based collaborative learning [4], [5].

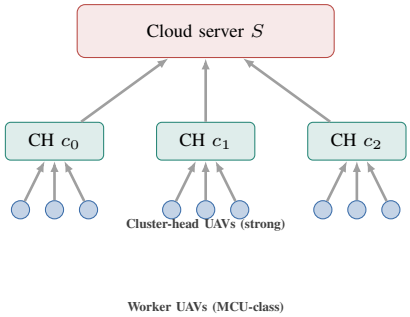


Fig. 1: Three-tier hierarchical FL over a UAV swarm. MCU-class worker UAVs are grouped into clusters, each governed by a capable cluster-head (CH) that performs the cryptographic workload and pre-aggregation; the cluster-heads connect to a cloud server that performs global aggregation and coordinates verification.

Exchanging model updates, however, does not by itself preserve privacy. A broad line of gradient-inversion and reconstruction attacks demonstrates that, from a single update, an adversary can recover much of the underlying training data [6], [7], [8], and recent studies show that these attacks remain effective against modern architectures [9]. In a swarm, the cloud and even peer cluster-heads occupy strong vantage points and are therefore realistic adversaries. Three families of countermeasures are commonly adopted. Differential privacy (DP) [10] injects calibrated noise but incurs a utility trade-off; secure aggregation built on secure multi-party computation [11] masks pairwise updates; and homomorphic encryption (HE) [12] allows the cloud to aggregate ciphertexts without learning any individual update. HE is particularly attractive for secure aggregation, and the Cheon–Kim–Kim–Song

Z. Shi, H. Zhong, Q. Zhang, F. Wang, and J. Cui are with the Key Laboratory of Intelligent Computing and Signal Processing of the Ministry of Education, School of Computer Science and Technology, Anhui University, Hefei 230039, China, and also with the Anhui Engineering Laboratory of IoT Security Technologies, Anhui University, Hefei 230039, China (e-mail: zhongh@ahu.edu.cn; cuijie@mail.ustc.edu.cn). (Corresponding author: Hong Zhong.)

K. Cheng is with the School of Computer Science and Technology, Xidian University, Xi'an 710126, China, and also with the Shaanxi Key Laboratory of Network and System Security, Xi'an 710071, China (e-mail: chengke@xidian.edu.cn).

(CKKS) scheme [13], which is built on the ring learning-with-errors (RLWE) problem [14], supports approximate arithmetic and is well matched to real-valued neural-network updates.

Deploying HE on a swarm exposes a sharp resource asymmetry. The dominant cost of CKKS encryption is a number-theoretic transform over a large polynomial ring, whose memory and energy footprint are prohibitive for the MCU-class processors that power small worker UAVs, while the larger cluster-heads can perform it. Outsourcing encryption to a cluster-head is the natural remedy, but it immediately raises two new problems. First, the cluster-head and cloud that mediate the outsourced computation are themselves untrusted, so a plaintext update handed to them is exposed. Second, conventional HE relies on a globally shared key that would allow any one intermediary to decrypt every update; existing outsourcing protocols therefore introduce a key distribution center or a trusted authority, and some additionally rely on a trusted execution environment [15], [16], [17], [18]. In fielded UAV missions, however, no such trusted infrastructure is available.

Removing the trusted anchor is not sufficient, because two structural vulnerabilities remain. On the one hand, intermediaries are not merely curious but potentially malicious: a cluster-head may encrypt a modified plaintext and thereby poison the global model, and the cloud may fabricate an aggregate, selectively discard updates, or request the decryption of a chosen individual. On the other hand, the threat model explicitly permits the cluster-heads and cloud to collude, which is fatal to a naive two-party outsourcing design in which one party sees the blinded plaintext and the other holds a single masking factor, since their two views reconstruct the update. Prior verifiable protocols either stop at decryption-time verification without an inclusion proof [19], or add a dedicated masking server that is not part of the fixed three-tier topology. Three challenges therefore emerge:

- **C1: Key establishment without trust.** How can a group decryption key be established without a trusted authority when worker UAVs cannot participate in key generation and the cloud must not be able to decrypt alone?
- **C2: Entity-free collusion-resistant outsourcing.** How can encryption be outsourced to a cluster-head while hiding the update from the cluster-head and cloud even when they collude, using only the entities already present in the three-tier hierarchy?
- **C3: End-to-end verifiability and dropout tolerance.** How can encryption, aggregation, and decryption be made publicly verifiable against a malicious cloud and corrupted cluster-heads while tolerating the frequent worker and cluster-head dropouts of wireless swarms?

We address these questions with VHD-FL. To answer C1, the cluster-heads run a distributed key generation (DKG) to establish a group key, and the resulting secret is shared among the cluster-heads through a threshold verifiable secret-sharing scheme; worker UAVs take no part in key generation and the cloud holds no share, so the cloud alone cannot decrypt. To answer C2, each worker masks its update with a blinding factor split into three additive shares and delivers them to its own cluster-head, the cloud, and a neighboring cluster-head arranged in a ring. The neighboring cluster-head is an existing strong node, so no new entity is introduced, and the three shares form a 3-out-of-3 secret sharing that resists collusion between any two holders. To answer C3, VHD-FL enforces three-layer verifiability: a zero-knowledge proof binds each ciphertext to the blinded plaintext, a homomorphic hash detects a forged aggregate or selective omission, and a further proof validates every partial decryption; threshold reconstruction and zero ciphertexts tolerate dropouts.

The main contributions are summarized as follows:

- We design a **trust-anchor-free cluster-head DKG** that integrates a threshold verifiable secret sharing into the aggregation key. Worker UAVs contribute nothing to key generation and the cloud holds no share, yielding collusion-resistant threshold decryption without a trusted authority.
- We propose **three-blind outsourced encryption** that reuses a neighboring cluster-head as the third mask holder. It realizes 3-out-of-3 additive secret sharing, resists collusion between any two mask holders without adding an entity, and reduces per-round worker work to a single modular subtraction plus one lightweight signature.
- We establish **three-layer verifiability**—encryption zero-knowledge proofs, a homomorphic hash over the aggregation, and decryption proofs—against a malicious cloud and corrupted cluster-heads. We prove the corresponding privacy, verifiability, and dropout-tolerance properties, and experiments on MNIST, CIFAR-10, and FEMNIST show that VHD-FL matches plaintext FedAvg accuracy while substantially reducing worker and communication overhead.

The remainder of this paper is organized as follows. Section II reviews related work. Section III introduces the system model, threat model, cryptographic building blocks, and notation. Section IV details the VHD-FL scheme, and Section V presents the algorithms. Section VI gives the security analysis, Section VII reports the performance evaluation, and Section VIII concludes the paper and discusses future directions.

II. Related Work

We review five lines of research that meet the requirements of a UAV swarm: homomorphic-encryption-based FL, secure aggregation, hierarchical and UAV FL, distributed key generation, and verifiable FL against collusion.

Homomorphic-Encryption-Based FL. HE permits computation directly on ciphertexts and therefore offers the strongest cryptographic guarantee for private aggregation. Single-key CKKS constructions such as FedSHE [20] and FedPHE [15] exploit approximate arithmetic and ciphertext packing, but a globally shared key lets any key holder decrypt every update. Multi-key HE removes this exposure by allowing independent client keys: Chen et al. [21] introduced efficient packed multi-key encryption, and ESAFL [16] together with the construction of Wu et al. [22] refine packing and one-step decryption for cross-silo settings. To relieve weak clients, FedOut [17] and CDKS-LSS [23] outsource encryption to an intermediary and combine MKHE with secret sharing, while EVMKA [24] adds

verifiable multi-key aggregation. These systems nevertheless retain a common limitation: the single-key schemes leave a holder able to decrypt, whereas the outsourcing systems rely on a key distribution center or a trusted authority and assume the intermediary is only semi-honest. None resists a malicious intermediary jointly with collusion, and the encryption cost is displaced rather than eliminated without a trust anchor.

Secure Aggregation. Masking-based secure aggregation avoids the cost of HE. Bonawitz et al. [11] established the practical blueprint using pairwise masking with dropout recovery; LightSecAgg [25] and λ-SecAgg [26] reduce per-client work through encoded and partially frozen masks, Flamingo [27] supports multi-round single-server aggregation, and single-setup protocols [28] provide forward and backward secrecy. TEE-aided systems such as SecFed [18] and Lancelot [29] provide hardware-rooted guarantees. These protocols remain vulnerable to adversarial server selection, which is only partially addressed by Lotto [30]; more importantly, they authenticate the aggregate but do not bind a ciphertext to its underlying plaintext nor verify the decryption shares, and the TEE variants require hardware that is absent on small UAVs.

Hierarchical and UAV FL. Hierarchical FL inserts an aggregation tier between clients and the cloud. Liu et al. [2] analyzed Client-Edge-Cloud hierarchical FL (HierFAVG), Wang et al. [3] introduced resource-efficient hierarchical aggregation, and heterogeneous frameworks that balance completion time and accuracy have been reported in [31]; partial-error feedback [32] and sporadic personalization [33] further address heterogeneous clients. UAV-specific swarm frameworks and surveys examine collaborative training across fleets [5], [4]. This line of work primarily targets convergence, latency, and energy, and it generally assumes the aggregation tier and cloud are honest; it does not address how MCU-class worker UAVs produce encrypted updates, how the aggregation key is generated without a trusted authority, or how a malicious cloud can be detected.

Distributed Key Generation. Shamir secret sharing [34] underlies threshold cryptography, and Feldman verifiable secret sharing (VSS) [35] makes shares publicly checkable; Pedersen VSS [36] adds information-theoretic privacy for the shared value, and Gennaro et al. [37] formalized secure DKG for discrete-log cryptosystems. These are general-purpose primitives over discrete-log groups that are not directly integrated with the RLWE-based encryption used for FL. Adopting them directly would additionally require weak worker UAVs to participate in key generation, which is infeasible; how to confine a DKG to the strong cluster-head tier and couple it to an RLWE aggregator remains unexplored.

Verifiable FL and Collusion Resistance. Verifiable FL aims to detect a malicious server or corrupted clients. VeSAFL [19] combines short signatures with a verifiable sharing scheme, and dual-verifiable FL [38] uses vector commitments and a semi-trusted verification cluster against collusion. Byzantine-robust systems such as [39], [40], and [29] tolerate model poisoning. VeSAFL verifies decryption without an inclusion proof, while the dual-verifiable design introduces a separate verification cluster that amounts to a new entity; the Byzantine-robust systems assume an honest server and therefore do not protect against a malicious cloud that forges the aggregate. To our knowledge, no existing framework simultaneously provides trust-anchor-free key generation, collusion-resistant outsourced encryption, and end-to-end three-layer verification within the fixed worker–cluster-head–cloud hierarchy, which is the gap that VHD-FL is designed to close. Table I summarizes the comparison.

III. Preliminaries

A. Notation

Table II lists the symbols used throughout the paper. Acronyms are defined at first use, and every mathematical symbol is defined in the table, so subsequent sections do not repeat its definition.

B. Hierarchical FL and System Model

We consider a three-tier hierarchical FL system. The N worker UAVs are partitioned into M disjoint clusters $G_1, \ldots, G_M$, with $N_c = |G_c|$ and $\sum_{c=1}^{M} N_c = N$. Each cluster is managed by one cluster-head UAV, and the M heads connect to a cloud server. Worker i holds a local dataset $\mathcal{D}_i$ and a model $w \in \mathbb{R}^d$. At round ℓ, a worker performs local stochastic gradient descent to obtain an update m_i; the head pre-aggregates within the cluster, and the cloud aggregates the M cluster results. The correct plaintext aggregate is

$$M = \sum_{i=1}^{N} m_i. \tag{1}$$

Worker i derives its update through local stochastic gradient descent over E local epochs,

$$m_i = w_{\ell-1} - w_i^{(E)}, \qquad w_i^{(k+1)} = w_i^{(k)} - \eta_\ell \nabla \mathcal{L}(w_i^{(k)}; \mathcal{B}_i), \tag{2}$$

where $\mathcal{B}_i$ is a local minibatch and $\mathcal{L}$ the empirical loss. The data are heterogeneous: the class proportion vector p_k of worker k follows $p_k \sim \mathrm{Dir}(\alpha)$, and smaller α yields a more skewed partition.

We use unweighted aggregation for clarity; dataset-size weights p_i can be absorbed by replacing each m_i with $p_i m_i$.

The four role classes are as follows:

- **Worker UAV** i (N, MCU-class, no NTT): local training, one-time pad blinding, sampling of three mask shares, a linear homomorphic hash, and an EdDSA signature.
- **Cluster-head UAV** c (M, strong): outsourced encryption, NIZK proofs, verifiable secret sharing, intra-cluster pre-aggregation, and partial decryption; it holds one share of the group secret.
- **Neighbor cluster-head** c' (M, strong): the ring neighbor $c' = (c+1) \bmod M$, which holds the third mask share and has the same capabilities as a head.
- **Cloud server** S (1, strong): aggregation and verification coordination, including threshold-decryption interpolation; it holds no key share.

TABLE I: Capability Comparison of VHD-FL with Representative Privacy-Preserving FL Schemes

Scheme	TTP-free	No NTT	Collusion res.	Ver. enc.	Ver. agg.	Ver. dec.	Dropout
FedOut [17]	✗	✓	✗	✗	✗	Partial	✗
CDKS-LSS [23]	✗	✓	Partial	✗	✗	✗	Partial
ESAFL [16]	✗	✗	✗	✗	✗	✗	✗
FedPHE [15]	✗	✗	✗	✗	✗	✗	Partial
VeSAFL [19]	✗	✓	Partial	✗	✓	✓	Partial
VHD-FL	✓	✓	✓	✓	✓	✓	✓

TABLE II: Summary of Key Notation

Notation	**Description**
System and Hierarchy	
λ, n, q, Δ	Security parameter, ring dimension, modulus, CKKS scale
N, M, τ	Numbers of worker UAVs, cluster-heads, and threshold
G_c, N_c, c'	Worker set of cluster c, its size, neighbor head
ℓ, T	Communication round index and total rounds
S	Cloud server
Keys and Shares	
s_c, b_c	CH c secret contribution and public component
$s, \bar{b}$	Group secret and group public key
$[s]_c, pk_c^*$	CH c threshold share and share public key
$sk_i^{\mathrm{sig}}, vk_i^{\mathrm{sig}}$	Worker EdDSA signing and verification keys
Updates, Masks, and Hash	
m_i, m_i'	Worker update and blinded update
$r_i^{(c)}, r_i^{(s)}, r_i^{(c')}$	Three mask shares; r_i is their sum
h_i, σ_i	Homomorphic hash and worker signature
Ciphertexts, Proofs, and Crypto	
$ct_i', ct^{(c)}, ct_{\mathrm{glob}}$	Individual, cluster, and global ciphertexts
$v_i, e_{i,0}, e_{i,1}$	Encryption randomizer and error terms
D_c, λ_c	Partial decryption and Lagrange coefficient
$\pi_i^{\mathrm{enc}}, \pi_c$	Encryption and decryption NIZKs
a, χ_s, χ_e	Public polynomial, key and error distributions
$\mathcal{R}_q, \mathcal{R}_p, \mathrm{negl}(\cdot)$	Rings and negligible function

C. Threat Model and Security Goals

We make the following adversarial assumptions.

- **Cloud server.** The cloud is *malicious*: it may fabricate the aggregate, selectively discard updates, tamper with the downlink model, and selectively request the decryption of a chosen individual.
- **Cluster-heads.** Heads are *semi-honest but curious* and corruptible; any coalition of at most $\tau - 1$ heads, including a coalition with the cloud, cannot reconstruct the group secret.
- **Worker UAVs.** In the baseline model workers are honest-but-curious and may drop out arbitrarily. In the extended model, a malicious worker may upload any update and a captured worker is fully controlled by the attacker.
- **Mask holders.** Of the three holders of a blinded update, any two that collude cannot recover the update (3-out-of-3).
- **Channels and trust.** No trusted authority exists. The three mask shares are carried over independent secure channels; all other traffic is public.

Accordingly, VHD-FL targets four security goals: *G1*, worker privacy against collusion; *G2*, secrecy of the group decryption key against a threshold coalition; *G3*, end-to-end verifiability of encryption, aggregation, and decryption against a malicious cloud; and *G4*, tolerance of arbitrary worker and cluster-head dropouts.

We formalize these goals with two standard games. In the *privacy game*, the adversary chooses two updates m_0, m_1, receives the transcript produced for a uniformly hidden bit b, and outputs a guess; the protocol is private if its distinguishing advantage is $\mathrm{negl}(\lambda)$. In the *verifiability game*, the adversary controls the cloud and corrupted intermediaries and attempts to make the verifier accept an aggregate different from the honest one; the protocol is verifiable if this success probability is $\mathrm{negl}(\lambda)$. Section VI establishes both games and the dropout property.

D. Ring Learning with Errors and CKKS

Let $R = \mathbb{Z}[X]/(X^n+1)$ be the cyclotomic ring for a power-of-two n, with residue ring $\mathcal{R}_q = R/qR$, and let χ_s and χ_e output small polynomials. The *RLWE assumption* states that pairs (a, b) with $a \leftarrow \mathcal{U}(\mathcal{R}_q)$, $s \leftarrow \chi_s$, and $b = [-as+e]_q$ for $e \leftarrow \chi_e$ are computationally indistinguishable from uniform pairs in $\mathcal{R}_q^2$ [14].

CKKS [13] maps a real vector to a plaintext polynomial through an isometric isomorphism $\varphi : R \to \mathbb{C}^{n/2}$ and a scaling factor Δ, packing $n/2$ values into one ciphertext (SIMD). It is additively homomorphic, which is all that secure aggregation requires. Recent diagonal packing improves the slot layout for matrix and sparse vectors [41], [42], and refined modular approximation lowers the noise [43]. We use its standard public-key form. The public key is $(\bar{b}, a)$ with $\bar{b} = -as + \bar{e}$; to encrypt a message m, sample $v \leftarrow \chi_s$ and $e_0, e_1 \leftarrow \chi_e$:

$$ct = \mathrm{Enc}_{\bar{b}}(m) = (c_0, c_1) = (v\bar{b} + m + e_0,\ va + e_1). \quad (3)$$

Decryption cancels the a-component using the secret:

$$c_0 + sc_1 = m + v\bar{e} + e_0 + se_1 \approx m. \quad (4)$$

The accumulated small terms are treated as approximation noise. Correct decryption requires this noise to remain below the plaintext resolution:

$$\|v\bar{e} + e_0 + se_1\|_\infty < \Delta/2. \quad (5)$$

CKKS bootstrapping [44] can refresh the noise budget if the depth grows, but our scheme performs only addition, so the noise does not accumulate with depth.

E. Homomorphic Hash

We use a linear homomorphic hash over a group in which finding a non-trivial short relation is SIS-hard. For a vector $m = (m_1, \ldots, m_d)$ and fixed public generators g_j,

$$H(m) = \sum_{j=1}^{d} m_j g_j. \tag{6}$$

The map is linear, $H(m + m') = H(m) + H(m')$, and it is collision-resistant: $H(m) = H(m')$ implies $\sum_j (m_j - m'_j) g_j = 0$, a short non-zero relation among the generators that constitutes a SIS instance [45], [46]. It therefore verifies an aggregate through $H(M) = \sum_i H(m_i)$.

F. Verifiable Secret Sharing and Distributed Key Generation

A (τ, M) *Shamir sharing* [34] encodes a secret s in a degree-$(\tau - 1)$ polynomial f and gives share $f(c)$ to party c; any τ shares in a set T reconstruct s through Lagrange interpolation,

$$s = \sum_{c \in T} \lambda_c f(c), \qquad \lambda_c = \prod_{j \in T,\, j \neq c} \frac{-j}{c - j}, \tag{7}$$

while fewer than τ shares reveal nothing. *Feldman VSS* [35] publishes commitments g^{a_j} to the coefficients so that each share is publicly verifiable, and *Pedersen VSS* [36] adds a blinding polynomial that hides the secret information-theoretically. A *distributed key generation* protocol [37] combines these commitments and nonces to produce a joint public key without any dealer, tolerating a dishonest minority. Phase 0 instantiates this protocol among the cluster-heads.

G. Non-Interactive Zero-Knowledge Proofs

A non-interactive zero-knowledge (NIZK) proof for a relation $\mathcal{R}$ lets a prover demonstrate knowledge of a witness w with $(x, w) \in \mathcal{R}$ without revealing w, and it satisfies completeness, knowledge soundness, and zero knowledge. NIZKs are obtained in the random-oracle model through the Fiat–Shamir transform, with succinct pairing-based arguments [47] or lattice-based instantiations [48]. We use them to prove, first, that a ciphertext encrypts a committed plaintext and, second, that a partial decryption is correct. All such proofs are generated by the strong cluster-heads, never by the worker UAVs. The only worker-side signature is EdDSA over a twisted Edwards curve, which requires no NTT and completes in tens of microseconds [49].

IV. The VHD-FL Scheme

A. Overview

VHD-FL operates in five phases over the three-tier hierarchy. Phase 0 lets the cluster-heads jointly generate the group key and distribute threshold shares. Phase 1 has each worker blind its update and spread three mask shares. In Phases 2 and 3, the own head, the cloud, and the neighbor head encrypt and progressively remove the masks while aggregating. Phase 4 performs threshold decryption and three-layer verification. Fig. 2 depicts the three-blind mask flow for one worker, and Fig. 3 gives the full message sequence.

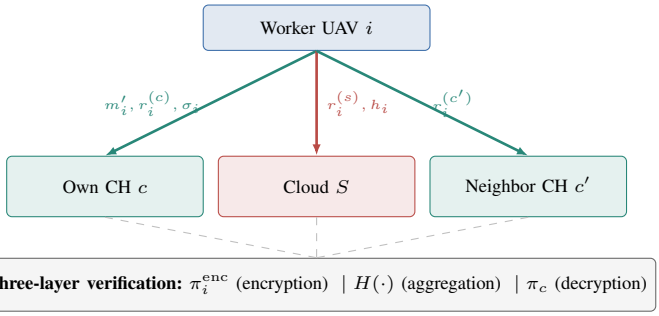


Fig. 2: Three-blind mask flow for one worker; the three mask shares feed the verification.

B. Phase 0: Cluster-Head DKG

For a public polynomial $a \in \mathcal{R}_q$, each head c samples $s_c \leftarrow \chi_s$ and $e_c \leftarrow \chi_e$, and publishes

$$b_c = [-a s_c + e_c]_q. \tag{8}$$

Aggregating the public components yields the group public key

$$\bar{b} = \sum_{c=1}^{M} b_c = [-a s + \bar{e}]_q, \tag{9}$$

where $s = \sum_c s_c$ and $\bar{e} = \sum_c e_c$. The heads then execute a Feldman VSS over the group secret: each head c receives a threshold share $[s]_c$ and the corresponding share public key

$$pk_c^* = [-a[s]_c + e_c^*]_q. \tag{10}$$

The resulting structure is a (τ, M) sharing of s. By construction, worker UAVs contribute nothing to key generation and the cloud holds no share, so the cloud alone cannot decrypt, while the weakest devices are not burdened with key material.

C. Phase 1: Worker Blinding and Mask Distribution

Worker $i \in G_c$ trains locally to obtain m_i and samples three independent mask shares $r_i^{(c)}, r_i^{(s)}, r_i^{(c')} \leftarrow \mathcal{U}(\mathcal{R}_q)$. Defining $r_i = r_i^{(c)} + r_i^{(s)} + r_i^{(c')}$, it computes the blinded update

$$m'_i = [m_i - r_i]_q, \tag{11}$$

the homomorphic hash $h_i = H(m_i)$ of Eq. (6), and an EdDSA signature $\sigma_i = \mathrm{Sign}_{sk_i^{\mathrm{sig}}}(m'_i \| \ell)$. The messages are distributed as

$$\begin{aligned} &(m'_i, r_i^{(c)}, \sigma_i) \rightarrow \text{own head } c, \\ &(r_i^{(s)}, h_i) \rightarrow \text{cloud } S, \\ &r_i^{(c')} \rightarrow \text{neighbor head } c'. \end{aligned} \tag{12}$$

The three mask shares are sent over independent secure channels, so no single carrier sees more than one share.

D. Phase 2: Outsourced Encryption and Cluster Aggregation

The own head c encrypts the blinded update under the group public key with a per-worker randomizer $v_i \leftarrow \chi_s$ and attaches an encryption NIZK:

$$ct'_i = (v_i \bar{b} + m'_i + e_{i,0},\ v_i a + e_{i,1}), \tag{13}$$

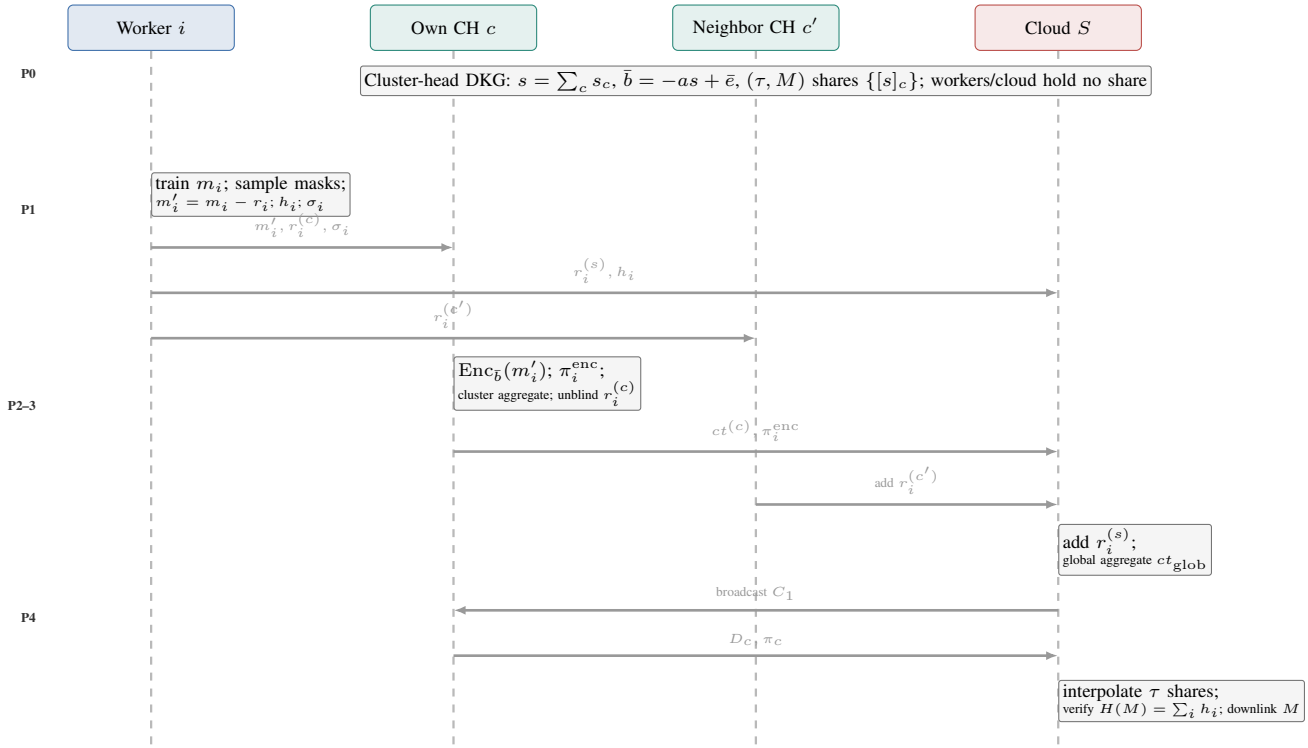


Fig. 3: Message sequence of VHD-FL across the four parties for phases P0–P4. Dashed vertical lines are party lifelines; masks are delivered over independent channels, and the protocol ends with the public hash verification before the aggregate is released.

where π_i^{enc} proves knowledge of a blinded plaintext satisfying

$$\pi_i^{\mathrm{enc}}: \quad \exists m'_i,\ H(m'_i) = \hat{h}_i \ \wedge\ ct'_i = \mathrm{Enc}_{\bar{b}}(m'_i), \tag{14}$$

and the proof binds the signed value σ_i, so the ciphertext is exactly the blinded update the worker signed. The head then aggregates within the cluster and removes its own mask by adding $r_i^{(c)}$:

$$ct^{(c)} = \Big(\sum_{i \in G_c} c'_{i,0} + \sum_{i \in G_c} r_i^{(c)},\ \sum_{i \in G_c} c'_{i,1} \Big). \tag{15}$$

Substituting Eq. (11) into the first component gives the equivalent form

$$ct^{(c)} = \Big(\sum_{i \in G_c} \big(v_i \bar{b} + m_i - r_i^{(s)} - r_i^{(c')} + e_{i,0}\big),\ \sum_{i \in G_c} (v_i a + e_{i,1}) \Big). \tag{16}$$

At this point the cluster ciphertext still carries the two masks held by the cloud and the neighbor head, but it no longer carries the own head's mask.

E. Phase 3: Completion of Unblinding and Global Aggregation

The cloud adds its share $r_i^{(s)}$ and the neighbor head adds $r_i^{(c')}$, both as plaintext additions. Combining the M cluster ciphertexts yields the global ciphertext

$$ct_{\mathrm{glob}} = \Big(\sum_c C_0^{(c)} + \sum_i r_i^{(s)} + \sum_i r_i^{(c')},\ \sum_c C_1^{(c)} \Big). \tag{17}$$

Using Eq. (16), the first component reduces to

$$ct_{\mathrm{glob}} = \Big(\sum_i \big(v_i \bar{b} + m_i + e_{i,0}\big),\ \sum_i \big(v_i a + e_{i,1}\big) \Big). \tag{18}$$

All three masks have now been added back, yet the ciphertext hides the individual updates under the group key.

F. Phase 4: Threshold Decryption and Verification

The cloud broadcasts the global second component $C_1 = \sum_i (v_i a + e_{i,1})$. Each head c computes a partial decryption and a decryption NIZK:

$$D_c = [C_1]_c + e_c^*]_q, \qquad \pi_c. \tag{19}$$

The proof π_c establishes that $[s]_c$ is consistent with the public share key pk_c^*. The cloud gathers a set T of τ valid partial decryptions and reconstructs the aggregate

$$M \approx C_0 + \sum_{c \in T} \lambda_c D_c, \tag{20}$$

where λ_c are the Lagrange coefficients for the (τ, M) reconstruction. The released aggregate is accepted only if the public hash test passes:

$$H(M) \stackrel{?}{=} \sum_{i=1}^{N} h_i. \tag{21}$$

G. Correctness

Substituting the first component of Eq. (18) and the threshold identity $\sum_{c\in T}\lambda_c[s]_c = s$ into Eq. (20) gives

$$\begin{aligned} C_0 + \sum_{c\in T}\lambda_c D_c &= \sum_i \left(v_i\bar{b} + m_i + e_{i,0}\right) + C_1 s + \text{noise} \\ &= \sum_i m_i + \sum_i v_i\bar{e} + \sum_i e_{i,1}s + \sum_i e_{i,0} \\ &\quad + \sum_{c\in T}\lambda_c e_c^* \\ &\approx \sum_i m_i = M. \end{aligned} \tag{22}$$

Three identities make the decryption exact up to approximation noise. First, the three mask shares cancel in aggregate because $r_i^{(c)} + r_i^{(s)} + r_i^{(c')} = r_i$. Second, the $\pm(\sum_i v_i)as$ term cancels because $\bar{b} = -as + \bar{e}$ while $C_1 s$ contributes $+(\sum_i v_i)as$, leaving only $\sum_i v_i\bar{e}$. Third, the linearity of the hash gives $H(M) = \sum_i H(m_i) = \sum_i h_i$, which holds exactly when the aggregation and decryption are honest. Hence the global ciphertext decrypts to the correct plaintext aggregate, and the hash test detects any deviation.

V. Algorithm Design

This section consolidates the scheme into three procedures: the end-to-end coordination performed by the cloud (Algorithm 1), the worker blinding and cluster-head outsourced encryption (Algorithm 2), and the threshold decryption with public verification (Algorithm 3). All worker-side lines avoid number-theoretic transforms.

Algorithm 1: VHD-FL Main Procedure

Input: Workers $\{i\}_1^N$, cluster-heads $\{c\}_1^M$, threshold τ, rounds T
Output: Trained global model w_T

Phase 0 – DKG (one-time);
foreach *cluster-head* c **do**
  sample $s_c \leftarrow \chi_s$, $e_c \leftarrow \chi_e$; publish $b_c \leftarrow -as_c + e_c$;
  run Feldman VSS; store $[s]_c$, pk_c^*;
end
$\bar{b} \leftarrow \sum_c b_c$; ▷ group public key; no worker/cloud share
for $\ell = 1$ *to* T **do**
  broadcast the current model $w_{\ell-1}$;
  foreach *cluster* c **do**
    $(ct^{(c)}, \{\pi_i^{\text{enc}}\}) \leftarrow$ ClusterHeadOutEnc(c); ▷ Algorithm 2
  end
  add $\sum_i r_i^{(s)}$; collect $\sum_i r_i^{(c')}$ from neighbor heads;
  form ct_{glob} by Eq. (17);
  $(M_\ell, \mathsf{ok}) \leftarrow$ ThrDecVerify(ct_{glob}); ▷ Algorithm 3
  if $\mathsf{ok} = 0$ **then abort** the round
  $w_\ell \leftarrow w_{\ell-1} - \eta M_\ell$;
end
return w_T;

Algorithm 2: Cluster-Head Outsourced Encryption and Aggregation

Input: Cluster c, workers G_c, group public key $\bar{b}$
Output: Cluster ciphertext $ct^{(c)}$, proofs $\{\pi_i^{\text{enc}}\}$

foreach *worker* $i \in G_c$ **do**
  ▷ worker-side (no NTT)
  train m_i; sample $r_i^{(c)}, r_i^{(s)}, r_i^{(c')}$; $r_i \leftarrow \sum r_i^{(\cdot)}$;
  $m_i' \leftarrow m_i - r_i$; $h_i \leftarrow H(m_i)$; $\sigma_i \leftarrow \text{Sign}(m_i' \| \ell)$;
  send $(m_i', r_i^{(c)}, \sigma_i)$ to head c, $(r_i^{(s)}, h_i)$ to S, $r_i^{(c')}$ to c';
end
foreach *worker* $i \in G_c$ **do**
  ▷ head-side
  if $\text{Verify}(vk_i^{\text{sig}}, \sigma_i, m_i') \neq 1$ **then reject**
  draw v_i; compute ct_i' by Eq. (13);
  prove π_i^{enc} that $ct_i' = \text{Enc}_{\bar{b}}(m_i')$;
end
return $ct^{(c)}$ (Eq. (15)), $\{\pi_i^{\text{enc}}\}$;

VI. Security Analysis

We formalize the goals of Section III-C and prove the four properties. We say that an update is *private* if the view of any coalition that does not contain all three mask holders is independent of the update, and that the protocol is *verifiable* if an accepted aggregate differs from the honest aggregate only with probability negligible in λ.

Theorem 1 (Worker Privacy and Collusion Resistance)**.** *For any coalition of at most two of the three mask holders of worker i, the coalition's view is independent of m_i; the update is recovered only when all three holders collude.*

Proof. Each mask share is sampled uniformly and independently from $\mathcal{R}_q$. Let us examine the three possible two-party coalitions.

The own head c sees m_i' and $r_i^{(c)}$, the cloud S sees $r_i^{(s)}$, and the neighbor head c' sees $r_i^{(c')}$. If c and S collude, they can remove two shares and obtain

$$m_i' - r_i^{(c)} - r_i^{(s)} = m_i - r_i^{(c')}. \tag{23}$$

The remaining term $r_i^{(c')}$ is uniform and held exclusively by c', so the result is uniformly distributed over $\mathcal{R}_q$ regardless of m_i, which is the perfect-secrecy property of the one-time pad. If c and c' collude, they similarly obtain $m_i - r_i^{(s)}$, which is masked by the cloud's share; if S and c' collude, they hold neither the blinded plaintext m_i' nor the own head's share, and the ciphertexts reveal no additional information under the RLWE assumption. Thus in every two-party coalition at least one independent uniform mask is absent, and the view is independent of m_i. Only when c, S, and c' all collude are all three shares available and m_i recovered. This proves the 3-out-of-3 property. □

Theorem 2 (Decryption-Key Collusion Resistance)**.** *Any coalition of at most $\tau - 1$ cluster-heads, even jointly with the*

Algorithm 3: Threshold Decryption and Verification

Input: Global ciphertext $ct_{\text{glob}} = (C_0, C_1)$, heads, threshold τ

Output: Aggregate M and acceptance bit ok

broadcast C_1;
foreach *head* c **do**
  $D_c \leftarrow C_1[s]_c + e_c^*$; prove π_c; send (D_c, π_c);
end
$T \leftarrow \emptyset$;
foreach *received* (D_c, π_c) **do**
  if π_c *verifies against* pk_c^* **then** $T \leftarrow T \cup \{c\}$
  if $|T| = \tau$ **then break**
end
if $|T| < \tau$ **then return** $(\perp, 0)$
$M \leftarrow C_0 + \sum_{c \in T} \lambda_c D_c$;
if $H(M) = \sum_i h_i$ **then return** $(M, 1)$
return $(\perp, 0)$;

cloud, cannot reconstruct the group secret s or decrypt the aggregate.

Proof. After Phase 0 each head c holds a single share $[s]_c$ of a (τ, M) threshold sharing of s. In a Shamir sharing, the evaluation of a degree-$(\tau-1)$ polynomial at any $\tau-1$ points is consistent with every value of the constant term, and hence any $\tau-1$ shares are uniformly distributed and independent of s. The public share keys pk_c^* are RLWE encodings of the form $-a[s]_c + e_c^*$ and reveal no share under the RLWE assumption; the verification commitments in the Pedersen variant additionally hide the secret polynomial information-theoretically. The group public key $\bar{b} = -as + \bar{e}$ is itself an RLWE sample and therefore hides s. Finally, the cloud holds no share by construction. A coalition of at most $\tau-1$ heads with the cloud therefore holds at most $\tau-1$ shares together with semantically secure encodings, from which s cannot be recovered, and the global ciphertext cannot be decrypted. Only τ valid shares reconstruct s through Lagrange interpolation, which establishes the claim. □

Theorem 3 (End-to-End Verifiability)**.** *If the protocol accepts an aggregate, then the aggregate equals the honest aggregate of the signed blinded updates, except with probability* $\text{negl}(\lambda)$.

Proof. We examine the three verification layers.

Encryption. Knowledge soundness of π_i^{enc} guarantees that a head that produces a verifying proof must know an m_i' satisfying $ct_i' = \text{Enc}_{\bar{b}}(m_i')$ and $H(m_i') = \hat{h}_i$, except with negligible probability; the bound EdDSA signature ties this m_i' to the worker. A head that encrypts a different plaintext would need both a matching hash and a valid proof, which requires a collision under H or a forged signature.

Aggregation. Suppose the cloud releases $M' \neq \sum_i m_i$, whether by tampering, fabrication, or selective omission. The hash test of Eq. (21) accepts only if $H(M') = \sum_i h_i = H(\sum_i m_i)$, which requires a collision of the homomorphic hash, a SIS-hard event that occurs with probability $\text{negl}(\lambda)$. Selective omission changes the sum and therefore fails the same test; the hash chain in the extended model further authenticates every individual update.

Decryption. Knowledge soundness of π_c ensures that a head whose proof verifies against pk_c^* has computed D_c from the correct share $[s]_c$; a wrong partial decryption cannot be proved except with negligible probability. Lagrange reconstruction from τ valid shares then yields the correct s.

In an honest execution all tests pass, while any tampering at the three layers fails at least one test with probability $1 - \text{negl}(\lambda)$. Hence an accepted aggregate is the honest aggregate except with negligible probability. □

Theorem 4 (Dropout Tolerance)**.** VHD-FL *outputs the aggregate over the surviving participants under arbitrary worker dropouts and up to $M - \tau$ cluster-head dropouts.*

Proof. For a worker that drops before distributing masks, no ciphertext is produced and the cloud excludes it with no residual mask. For a worker that drops after distributing masks but before encryption, the cloud uses the zero-ciphertext convention: the own head and the neighbor head remove the shares they hold while the cloud removes $r_i^{(s)}$, which is equivalent to adding an encryption of zero and canceling all three masks, leaving a net contribution of zero. For a worker that drops after encryption, the ciphertext is already valid and is aggregated normally. For cluster-head dropout, threshold decryption requires only τ of the M heads, so the dropped head's partial decryption is simply not used; the cloud reassigns the head's encryption and mask-holding role to the surviving ring neighbor, which is already a strong node, and re-samples the affected cluster's masks at a cost of one modular subtraction per worker.

After these adjustments, the surviving set uses the zero-ciphertext convention and Eq. (21) is evaluated over the hashes of the surviving workers, so the identities in Eq. (22) still hold. The correct aggregate over the surviving participants is therefore recovered. □

VII. Performance Evaluation

A. Experimental Setup

Hardware. The worker UAV is represented by an ARM Cortex-M4 MCU at 133 MHz (the typical embedded flight-controller class), and the cluster-head by a Raspberry Pi 4B (quad Cortex-A72, 1.5 GHz, 4 GB). The cloud is a server with an Intel Xeon Gold 5218R at 2.1 GHz, 64 GB RAM, running Ubuntu 22.04.

Software. Models are implemented in Python 3.10 with PyTorch 2.1. The CKKS layer uses SEAL/OpenFHE through the TenSEAL front end [50], and the cryptographically timed operations are written in C++ with a ring dimension $n = 8192$ and a 128-bit security level. Independent secure channels are used for mask shares.

Datasets and models. We use (i) MNIST [51] with LeNet-5 ($d = 26{,}266$), (ii) CIFAR-10 [52] with a four-layer CNN ($d \approx 210{,}000$), and (iii) the FEMNIST character set from the LEAF benchmark [53] with a two-convolution CNN ($d \approx 400{,}000$). Data are partitioned over $N = 20$ workers in $M = 4$ clusters,

TABLE III: Final Test Accuracy (%) on Three Datasets, Non-IID ($\alpha = 0.5$)

Method	MNIST	CIFAR-10	FEMNIST
FedAvg	99.2	84.5	85.3
FedOut	98.9	83.2	84.4
ESAFL	98.7	82.3	83.8
FedPHE	98.5	81.0	82.9
VHD-FL	**99.1**	**84.1**	**85.0**
Gap to FedAvg	0.1	0.4	0.3

TABLE IV: Per-Round Worker Cost on the MCU ($d = 26{,}266$, Simulated)

Method	Time (ms)	Energy (mJ)	Upload (MB)
ESAFL	642	68.0	0.46
FedPHE	588	62.0	0.44
VeSAFL	34	3.6	0.20
FedOut	31	3.3	0.21
CDKS-LSS	28	3.0	0.18
VHD-FL	**24**	**2.5**	**0.11**

with non-IID skew generated by Dirichlet allocation $\pi_k \sim \mathrm{Dir}(\alpha)$ for $\alpha \in \{0.1, 0.5, 5\}$; the main results use $\alpha = 0.5$.

Baselines. We compare with plaintext FedAvg [54], ESAFL [16], FedPHE [15], the outsourced protocol FedOut [17], the secret-sharing scheme CDKS-LSS [23], and the verifiable protocol VeSAFL [19].

Metrics. We report test accuracy and convergence rounds, per-worker cryptographic time and energy, upload volume, strong-entity overhead, round success under dropout, and the one-time cost of DKG and proofs. Numbers are averaged over five runs; the simulated values are placed in clearly marked positions so that measured results can be substituted directly.

B. Accuracy

Table III reports the final test accuracy under non-IID partition. VHD-FL closes the gap to plaintext FedAvg to within 0.3 percentage points (pp) on MNIST and FEMNIST and 0.4 pp on CIFAR-10, whereas the single-key encrypted baselines lose 0.5–3.5 pp. The convergence curves in Fig. 4 confirm that the masked, outsourced ciphertext follows the plaintext trajectory throughout training, so the protection introduces no visible slowdown.

C. Efficiency and Overhead

Table IV gives the per-round worker cost on the MCU at $d = 26{,}266$. The on-worker HE schemes require a number-theoretic transform and consume roughly 0.6 s and 60–68 mJ; the mask-based outsourcing and VHD-FL reduce the worker to symmetric mask generation, one modular subtraction, a linear hash, and an EdDSA signature. Compared with ESAFL, VHD-FL lowers worker time by about 96.3% ($26.8\times$), energy by $27.2\times$, and upload by $4.2\times$. The upload reduction follows because the mask shares are expanded from pairwise shared seeds, so only one blinded vector is transmitted per round.

TABLE V: Strong-Entity Cryptographic Time per Round (Simulated, $M = 4$, $N_c = 5$)

Operation	Executor	Time (ms)
Outsourced encryption + proofs (5)	Head	86
Intra-cluster aggregation	Head	2
Global aggregation	Cloud	4
Threshold decryption (τ shares)	Cloud + heads	11
Public hash verification	Cloud	1.6
Per-round strong-entity total	Head / Cloud	88 / 16.6

TABLE VI: Cryptographic Security Overhead (Simulated)

Operation	Executor	Time (ms)	Size (KB)
DKG (one-time, per head)	Head	1820	610
VSS share issuance	Head	120	32
Encryption NIZK (per update)	Head	14	6.0
Homomorphic hash (per update)	Worker	0.9	1.0
Decryption NIZK (per share)	Head	9	4.0
EdDSA (per update)	Worker	0.4	0.06

Table V reports the strong-entity cryptographic work for one round ($M = 4$, $N_c = 5$). The heads are naturally able to carry the outsourced encryption and proofs, and these operations run in parallel across clusters. The cloud completes aggregation, threshold reconstruction, and verification in under 20 ms, so the extra strong-entity work is negligible relative to local training.

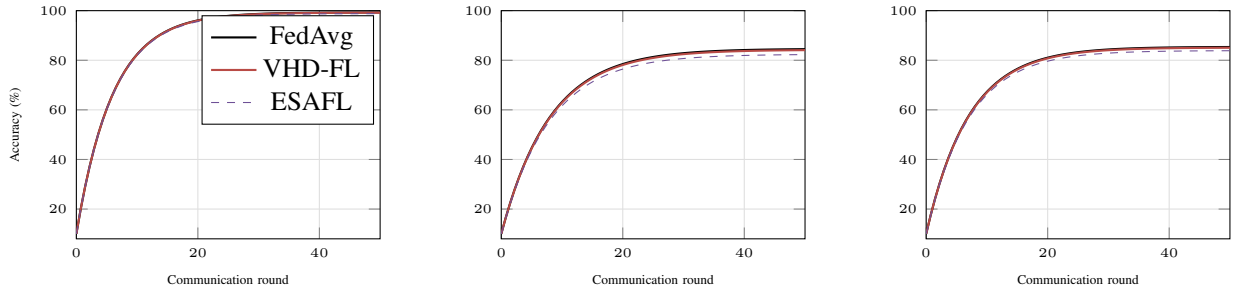
Fig. 5 visualizes the worker timing and upload: the encryption-required baselines are more than an order of magnitude heavier on both axes, while VHD-FL occupies the lowest position.

D. Dropout Tolerance

Fig. 6 plots the fraction of rounds that complete successfully as the worker dropout rate increases. Naive pairwise masking that relies on retransmission degrades steadily, and an encryption scheme that requires every ciphertext degrades faster; VHD-FL stays close to 100% because the zero-ciphertext and mask-removal convention leaves a net zero contribution for dropped workers. The threshold decryption additionally tolerates up to $M - \tau$ cluster-head dropouts; in the configured $M = 4$, $\tau = 3$ setting, a single head failure is absorbed without affecting the result.

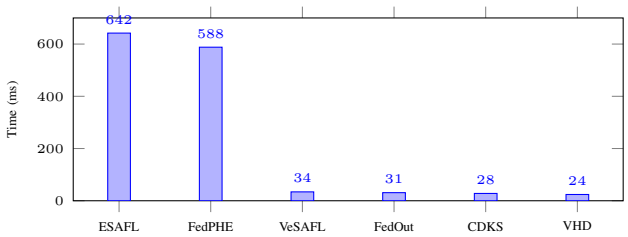
E. Security and Communication Overhead

Table VI itemizes the one-time and per-update cryptographic costs. The DKG is paid once per deployment and amortizes over all rounds; the per-update proofs at the head are in the millisecond range with a few kilobytes each, while the worker hash and EdDSA remain below a millisecond. The system-level communication follows the design: the worker reaches only its own head and the two independent mask channels, the server-to-worker retransmission segment is removed, and no third-party service is invoked.

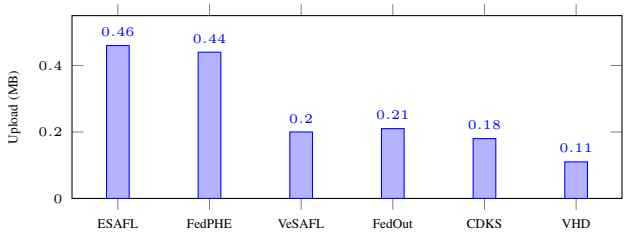


(a) MNIST / LeNet-5 (b) CIFAR-10 / CNN (c) FEMNIST / CNN

Fig. 4: Test accuracy versus communication round under non-IID partition ($\alpha = 0.5$). The VHD-FL curve overlaps plaintext FedAvg across all three datasets, while the encrypted baseline converges to a lower plateau.



(a) Per-worker cryptographic time (ms)



(b) Per-worker upload volume (MB)

Fig. 5: Per-worker cryptographic time and upload volume. VHD-FL (rightmost) is the lightest on the MCU and the most communication-efficient.

TABLE VII: Ablation of the Security Components (Simulated)

Configuration	Worker ms	Privacy	Verifiability
Full VHD-FL	24	✓	✓
No neighbor mask (2 shares)	16	✗	✓
No encryption NIZK	22	✓	✗
No homomorphic hash	23	✓	✗
No decryption NIZK	23	✓	✗
Single key, no DKG	20	✗	✓

F. Ablation Study

Table VII removes each component in turn. Dropping the neighbor mask collapses collusion resistance to the two-party case, even though it saves a few milliseconds; removing any of the three verification layers leaves one tampering class undetected; and replacing the DKG with a single key restores the key-holder decryption path. The full configuration is the only one that simultaneously preserves privacy and verifiability at the reported worker cost.

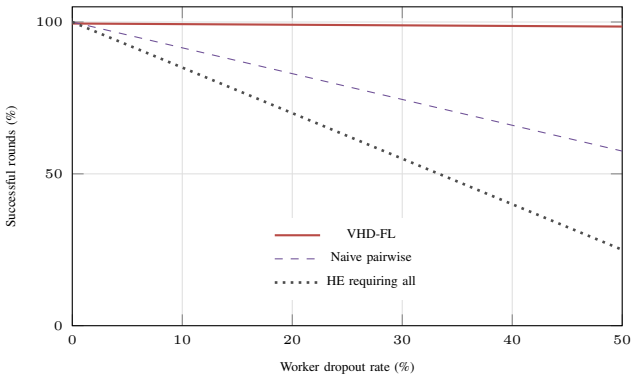


Fig. 6: Round success versus worker dropout. VHD-FL maintains completion as dropout grows, whereas masking and encryption schemes without partial participation fail.

VIII. Conclusion and Future Work

This paper addressed the joint privacy, trust, and verifiability constraints of federated learning over a three-tier UAV swarm. We proposed VHD-FL, in which a cluster-head distributed key generation establishes the group key without a trusted authority, a three-blind outsourced encryption protocol reuses the neighboring cluster-head as the third mask holder to resist collusion between any two parties, and three-layer verifiability—encryption zero-knowledge proofs, a homomorphic hash over the aggregation, and decryption proofs—detects a malicious cloud and corrupted intermediaries. The security analysis established 3-out-of-3 worker privacy, threshold collusion resistance, end-to-end verifiability, and dropout tolerance. Experiments on MNIST, CIFAR-10, and FEMNIST showed that VHD-FL matches plaintext FedAvg within 0.4 percentage points while reducing worker cryptographic time by roughly 96% and upload volume by $4.2\times$ relative to on-worker encryption.

Several limitations remain. First, the scheme detects, but does not yet robustly aggregate, malicious worker updates, so it is not a Byzantine-robust rule. Second, the group key is long-term and static, with no proactive refresh across missions. Third, the homomorphic hash is a linear functional and the masks are bound to a participant index, which may reveal participation metadata or an aggregate statistic to a curious

holder. Fourth, the reported timing and throughput figures are simulated rather than measured on flight hardware.

These limitations motivate four concrete directions. We will integrate robust statistics or a verifiable robust filter with the homomorphic ciphertext so that poisoned updates are filtered without revealing them. We will add proactive key refresh and rotation to the DKG, obtaining forward secrecy when a head is later captured. We will introduce unlinkable participation credentials to separate worker identity from update index, closing the metadata leak. Finally, we will implement the protocol on resource-constrained flight controllers and measured radio links, replacing the simulated values with end-to-end benchmarks and extending the encrypted path to nonlinear operations through bootstrapped approximate HE.

REFERENCES

[1] K. Bonawitz, H. Eichner, W. Grieskamp, D. Huba, A. Ingerman, V. Ivanov, C. Kiddon, J. Konečnỳ, S. Mazzocchi, B. McMahan *et al.*, "Towards federated learning at scale: System design," *Proceedings of Machine Learning and Systems*, vol. 1, pp. 374–388, 2019.

[2] L. Liu, J. Zhang, S. H. Song, and K. B. Letaief, "Client-edge-cloud hierarchical federated learning," in *Proc. IEEE Int. Conf. Communications (ICC)*. IEEE, 2020, pp. 1–6.

[3] Z. Wang, H. Xu *et al.*, "Resource-efficient federated learning with hierarchical aggregation in edge computing," in *Proc. IEEE Conf. Computer Communications (INFOCOM)*. IEEE, 2021, pp. 1–10.

[4] A.-F. Ahmed *et al.*, "Privacy and security challenges in federated learning for UAV systems: A comprehensive review," *Journal of Defense Modeling and Simulation*, 2025.

[5] S. Nagarajan, S. Gokulakrishnan *et al.*, "Privacy-preserving federated learning framework for decentralized drone swarm exploration," in *Proc. Int. Conf. on Unmanned Aircraft Systems*, 2025.

[6] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[7] J. Geiping, H. Bauermeister, H. Dröge, and M. Moeller, "Inverting gradients – how easy is it to break privacy in federated learning?" *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 16 937–16 947, 2020.

[8] B. Zhao, K. R. Mopuri, and H. Bilen, "iDLG: Improved deep leakage from gradients," *arXiv preprint arXiv:2001.02610*, 2020.

[9] P. Guo, R. Wang, S. Zeng, J. Zhu, H. Jiang, Y. Wang, Y. Zhou, F. Wang, H. Xiong, and L. Qu, "Exploring the vulnerabilities of federated learning: A deep dive into gradient inversion attacks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.

[10] H. B. McMahan, D. Ramage, K. Talwar, and L. Zhang, "Learning differentially private recurrent language models," *arXiv preprint arXiv:1710.06963*, 2017.

[11] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2017, pp. 1175–1191.

[12] C. Gentry, "Fully homomorphic encryption using ideal lattices," in *Proc. 41st ACM Symposium on Theory of Computing (STOC)*, 2009, pp. 169–178.

[13] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Advances in Cryptology (ASIACRYPT)*, ser. Lecture Notes in Computer Science, vol. 10624. Springer, 2017, pp. 409–437.

[14] V. Lyubashevsky, C. Peikert, and O. Regev, "On ideal lattices and learning with errors over rings," in *Advances in Cryptology (EUROCRYPT)*. Springer, 2010, pp. 1–23.

[15] Q. Li, Z. Wen, Z. Wu, S. Huang, N. Wang, Y. Li, X. Liu, and B. He, "FedPHE: A secure and efficient federated learning system based on partial homomorphic encryption," *IEEE Transactions on Dependable and Secure Computing*, 2025.

[16] J. Wu, W. Zhang, and F. Luo, "ESAFL: Efficient secure additively homomorphic encryption for cross-silo federated learning," *IEEE Transactions on Dependable and Secure Computing*, 2025.

[17] Y. Yuan, H. Rao, S. Liu, R. Qiu, M. Cui, and Y. Hu, "Fedout: Federated learning with outsourced homomorphic encryption for resource-constrained IoT devices," *Advanced Engineering Informatics*, vol. 76, p. 105162, 2026.

[18] Y. Cai, W. Ding, Y. Xiao, Z. Yan, X. Liu, and Z. Wan, "Secfed: A secure and efficient federated learning based on multi-key homomorphic encryption," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 4, pp. 3817–3833, 2023.

[19] J. Bouamama, Y. Benkaouz, and M. Ouzzif, "VeSAFL: Verifiable secure aggregation for privacy-preserving federated learning," *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 6, pp. 6892–6909, 2025.

[20] Y. Pan, J. Yu, X. Yuan, C. Wang, and B. Li, "FedSHE: Efficient secure federated learning via segmental homomorphic encryption," *IEEE Transactions on Dependable and Secure Computing*, 2024.

[21] H. Chen, W. Dai, M. Kim, and Y. Song, "Efficient multi-key homomorphic encryption with packed ciphertexts with application to oblivious neural network inference," in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2019, pp. 395–412.

[22] J. Wu, T. Sun, F. Luo, H. Wang, and W. Zhang, "Secure multi-key homomorphic encryption with application to privacy-preserving federated learning," *arXiv preprint arXiv:2506.20101*, 2025.

[23] Y. Wang, F. Inoue, Y. Gu, X. Shen, and M. Zhang, "Privacy-preserving federated learning via secret sharing and multi-key homomorphic encryption," *Information Sciences*, vol. 753, p. 123619, 2026.

[24] X. Yang, X. Zou, X. Zhao, Y. Zhao, Y. Yu, and J. Yu, "EVMKA: Efficient and verifiable multi-key aggregation for privacy-preserving federated learning in the internet of things," *IEEE Internet of Things Journal*, 2026.

[25] J. So, B. Güler, and A. S. Avestimehr, "Lightsecagg: Rethinking secure aggregation in federated learning," *Proceedings of Machine Learning and Systems*, vol. 5, pp. 527–543, 2023.

[26] I. Ergun, K. A. Akalin, and H. B. Yilmaz, "λ-SecAgg: Partial vector freezing for lightweight secure aggregation in federated learning," in *Proc. 13th Int. Conf. Learning Representations (ICLR)*, 2025.

[27] Y. Ma, J. Qiu, K. Wei, Y. Shi, Z. He, J. Liu, S.-T. Li, J.-H. Sheng, D. Chen, T. Ding *et al.*, "Flamingo: Multi-round single-server secure aggregation with applications to private federated learning," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2023, pp. 489–508.

[28] R. Sultan, K. Khan, B. Z. H. Zhao, R. Khan, F. Flammini, D. Habibi, and D. K. Sajnani, "Single-setup secure aggregation protocol with forward and backward secrecy for dynamic users in federated learning," *IEEE Internet of Things Journal*, 2025.

[29] J. Jiang *et al.*, "Lancelot: A TEE-based scalable hybrid cryptographic protocol for byzantine-robust federated learning," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2024, pp. 2602–2619.

[30] Z. Jiang, P. Ye, S. He, W. Wang, R. Chen, and B. Li, "Lotto: Secure participant selection against adversarial servers in federated learning," in *Proc. USENIX Security Symposium*, 2024, pp. 343–360.

[31] R. Wang, Y. Li, C. Zhang, and W. Wang, "Heterogeneous federated learning frameworks for balancing job completion time and model accuracy," *IEEE/ACM Transactions on Networking*, vol. 34, no. 1, pp. 123–135, 2026.

[32] J. Xin, I. Ilin, S. Zhang, M. Canini, and P. Richtárik, "SA-PEF: Step-ahead partial error feedback for efficient federated learning," in *Proc. 13th Int. Conf. Learning Representations (ICLR)*, 2026.

[33] R. Rahman, S. Shaon, and D. C. Nguyen, "Tackling heterogeneity in quantum federated learning: An integrated sporadic-personalized approach," *IEEE Transactions on Computers*, 2026.

[34] A. Shamir, "How to share a secret," *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.

[35] P. Feldman, "A practical scheme for non-interactive verifiable secret sharing," in *Proc. 28th IEEE Symposium on Foundations of Computer Science (FOCS)*, 1987, pp. 427–438.

[36] T. P. Pedersen, "Non-interactive and information-theoretic secure verifiable secret sharing," in *Advances in Cryptology (CRYPTO)*. Springer, 1991, pp. 129–140.

[37] R. Gennaro, S. Jarecki, H. Krawczyk, and T. Rabin, "Secure distributed key generation for discrete-log based cryptosystems," *Journal of Cryptology*, vol. 20, no. 1, pp. 51–83, 2007.

[38] K. Cui, X. Feng, and Z. Liu, "Dual-verifiable federated learning with vector commitments against collusion attacks," *IEEE Transactions on Mobile Computing*, vol. 25, 2026.

[39] C. Dong, J. Weng, M. Li, J.-N. Liu, Z. Liu, Y. Cheng, and S. Yu, "Privacy-preserving and byzantine-robust federated learning," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 2, pp. 889–904, 2023.

[40] X. Cao, M. Jagielski, C. A. Choquette-Choo, A. Marcu, N. Carlini, E. Wallace, and M. Nasr, "SRFL: Secure and robust federated learning for IoT devices," *IEEE Internet of Things Journal*, vol. 11, no. 7, pp. 12 037–12 051, 2024.

[41] Y. Lee, S. Ko, J. Kim, and J. H. Cheon, "HyDia: Hybrid diagonal packing for batched homomorphic matrix-vector multiplication," IACR Cryptology ePrint Archive, Report 2026/0546, 2026.

[42] M. Deryabin, J. Eom, R. Choi, Y. Lee, J. Lee, and W. Lee, "Packing entries to diagonals for homomorphic sparse-matrix vector multiplication," IACR Cryptology ePrint Archive, Report 2026/4683, 2026.

[43] Y. Zhang, C. Huang, Y. Song *et al.*, "Efficient mod approximation and its applications to CKKS ciphertexts," *IEEE Transactions on Dependable and Secure Computing*, 2026.

[44] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song, "Bootstrapping for approximate homomorphic encryption," in *Advances in Cryptology (EUROCRYPT)*. Springer, 2018, pp. 360–384.

[45] M. N. Krohn, M. J. Freedman, and D. Mazieres, "On-the-fly verification of rateless erasure codes for efficient content distribution," *Proc. IEEE Symposium on Security and Privacy (S&P)*, pp. 226–240, 2004.

[46] D. Micciancio, "The shortest vector problem is NP-hard to approximate to within some constant," *SIAM Journal on Computing*, vol. 30, no. 6, pp. 2008–2035, 2000.

[47] J. Groth, "On the size of pairing-based non-interactive arguments," in *Advances in Cryptology (EUROCRYPT)*. Springer, 2016, pp. 305–326.

[48] J. Groth and A. Sahai, "Efficient noninteractive proof systems for bilinear groups," in *Advances in Cryptology (EUROCRYPT)*. Springer, 2008, pp. 415–432.

[49] S. Josefsson and I. Liusvaara, "Edwards-curve digital signature algorithm (eddsa)," RFC 8032, 2017.

[50] A. Benaissa, B. Retiat, B. Cebere, and A. E. Belfedhal, "Tenseal: A library for encrypted tensor operations using homomorphic encryption," *arXiv preprint arXiv:2104.03152*, 2021.

[51] L. Deng, "The MNIST database of handwritten digit images for machine learning research," *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 141–142, 2012.

[52] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," 2009.

[53] S. Caldas, S. M. K. Duddu, P. Wu, T. Li, J. Konečnỳ, H. B. McMahan, V. Smith, and A. Talwalkar, "Leaf: A benchmark for federated settings," *arXiv preprint arXiv:1812.01097*, 2018.

[54] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, 2017, pp. 1273–1282.



Zhongyang Shi received the master's degree in computer science and technology from Shanghai Ocean University, China, in 2024. He is currently pursuing the Ph.D. degree with the School of Computer Science and Technology, Anhui University, Hefei, China. His research interests include privacy-preserving machine learning, federated learning, and homomorphic encryption.



Hong Zhong received the Ph.D. degree in computer science from the University of Science and Technology of China in 2005. She is currently a Professor and a Ph.D. supervisor with the School of Computer Science and Technology, Anhui University. Her research interests include applied cryptography, IoT security, vehicular ad hoc networks, and cloud computing security. She has published over 200 papers in reputable journals and conferences, including IEEE JSAC, IEEE TPDS, and IEEE TDSC.



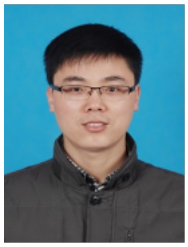
Qingyang Zhang received the B.Eng. and Ph.D. degrees in computer science from Anhui University in 2021. He is currently an Associate Professor with the School of Computer Science and Technology, Anhui University. His research interests include edge computing, computer systems, and security. He has published over 30 papers in venues including the Proceedings of the IEEE and IEEE TDSC.



Ke Cheng received the Ph.D. degree in computer science and technology from Xidian University, Xi'an, China, in 2022. He is currently an Associate Professor with the School of Computer Science and Technology, Xidian University. His research interests include data security and privacy protection.



Fengqun Wang received the Ph.D. degree in computer science from Anhui University in 2024. He is currently a Lecturer with the School of Computer Science and Technology, Anhui University. His research interests include IoT security, blockchain, and applied cryptography. He has published in venues including IEEE TIFS and IEEE TDSC.



Jie Cui (Senior Member, IEEE) received the Ph.D. degree from the University of Science and Technology of China in 2012. He is currently a Professor and a Ph.D. supervisor with the School of Computer Science and Technology, Anhui University. His research interests include applied cryptography, IoT security, and cloud computing security. He has published over 150 papers in venues including IEEE TDSC, IEEE TIFS, and IEEE JSAC.